

SYS-304 — Milestone 3: Scaling & Optimization

Benchmark Results and Architecture Diagram — Digit Recognizer Service

1. Benchmark Results

Two independent sets of optimizations were benchmarked: **model-level** techniques (Week 5) and **infrastructure-level** techniques (Week 6), both compared against the naive Phase 2 baseline.

Variant	Latency (mean)	Speedup	RSS Memory	Disk size	Accuracy
Naive PyTorch (Phase 2)	104.5 ms	1.0x	94.6 MB	44.8 MB	97.52%
ONNX FP32	19.3 ms	5.4x	79.0 MB	44.7 MB	97.52%
ONNX INT8 (dynamic quant.)	49.1 ms	2.1x	36.7 MB	11.2 MB	not measured
Distilled student	1.6 ms	67.3x	1.3 MB	39.4 KB	97.06%

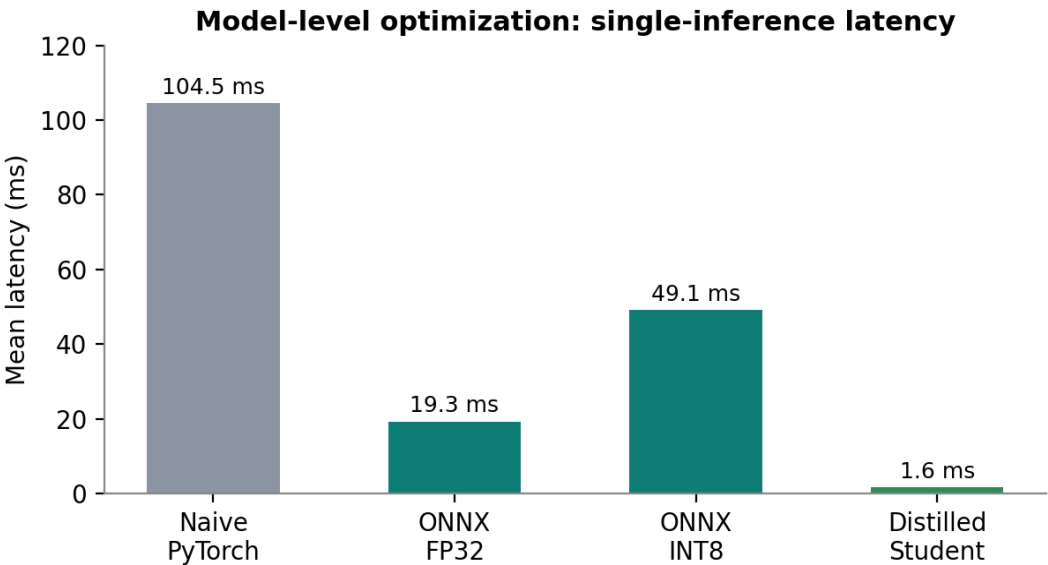


Figure 1. Single-inference latency across model-level variants.

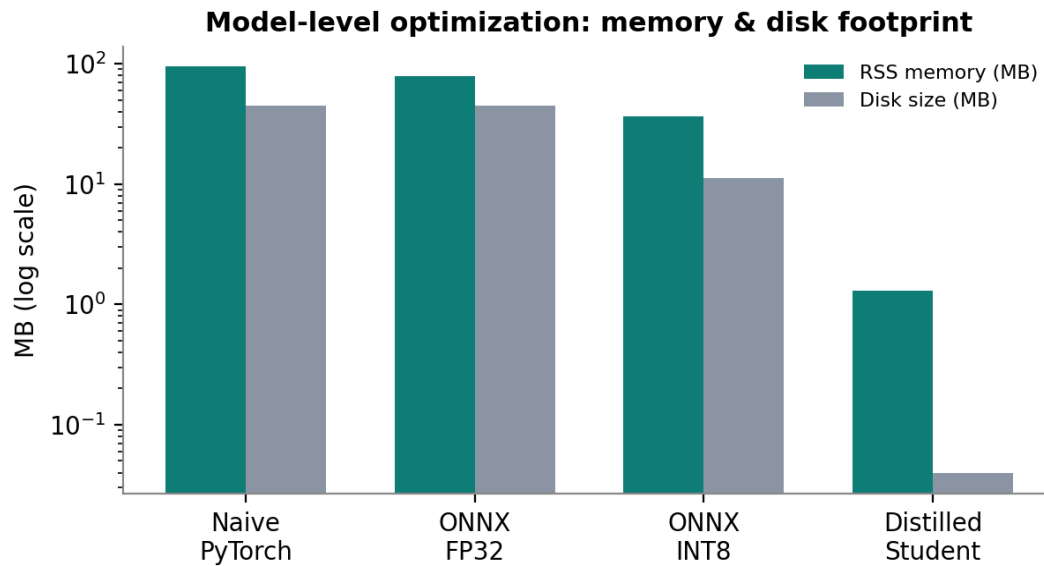


Figure 2. Memory (RSS) and on-disk size, log scale, across model-level variants.

The distilled student dominates every efficiency metric — 67x faster and over 1,000x smaller than the naive baseline — at a cost of under half a percentage point of accuracy. ONNX export alone gives a 5.4x speedup with zero accuracy cost. INT8 quantization gives the largest disk-size reduction (4x) but a smaller latency win, reflecting the runtime cost of computing activation ranges dynamically.

2. Infrastructure-level optimization (Week 6)

Configuration	Total wall time	Mean per-request	Throughput
Baseline (blocking, single-threaded)	689.0 ms	620.8 ms	29.0 req/s
Thread-pool offload only	1042.6 ms	954.2 ms	19.2 req/s
Dynamic batching (max 4 / 2ms window)	577.2 ms	449.1 ms	34.6 req/s

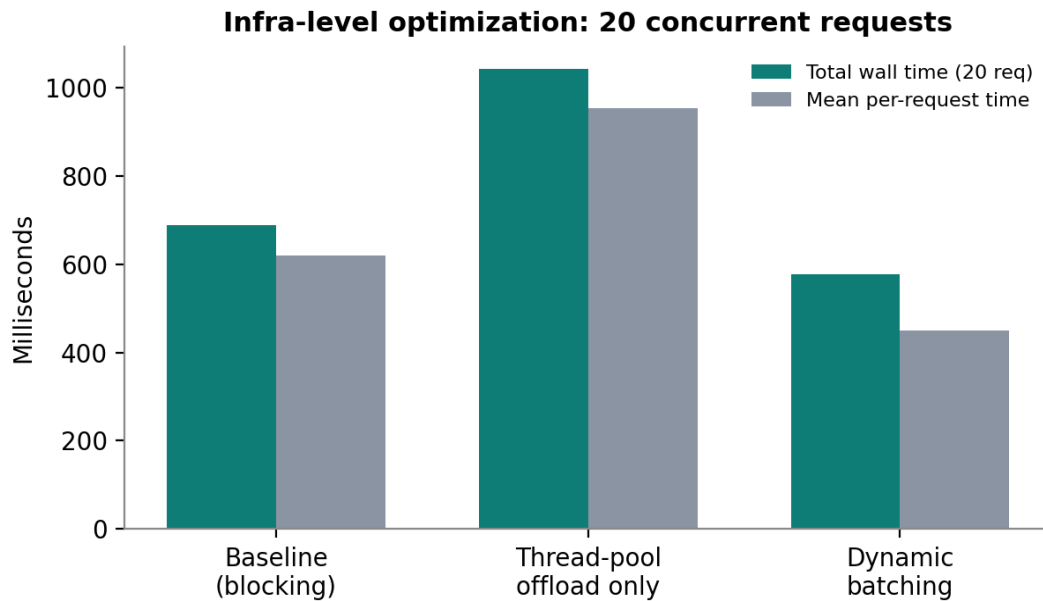


Figure 3. Total and mean request latency under 20 concurrent requests.

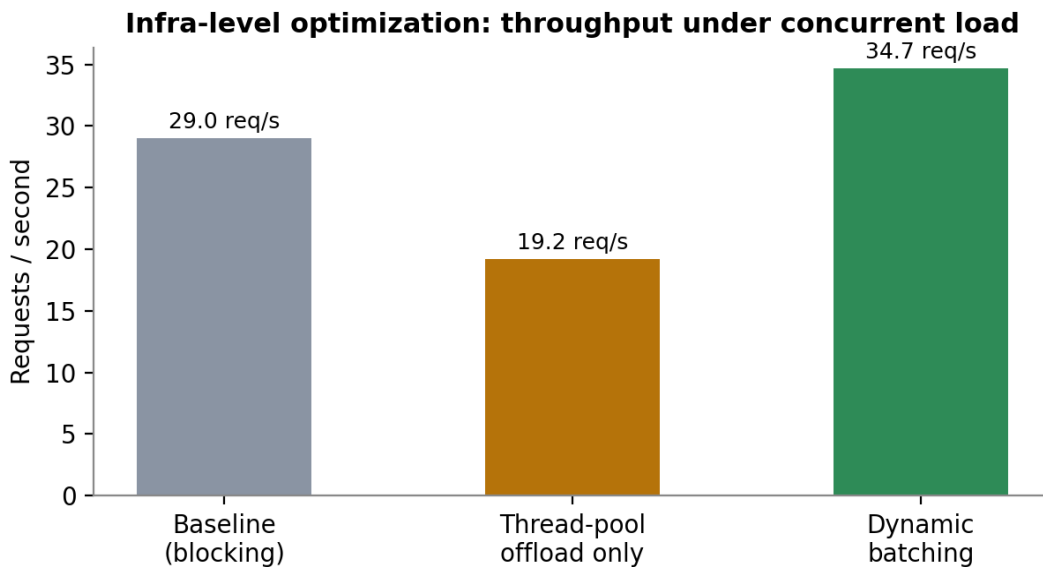


Figure 4. Derived throughput (20 requests / total wall time) under concurrent load.

Thread-pool offloading alone made concurrent performance worse (1042.6 ms vs. 689 ms baseline): the overhead of dispatching each request to its own worker thread outweighed the benefit of not queueing on a task this fast. Dynamic batching resolved this by grouping requests before touching threads at all — 20 requests became 6 batches, so the fixed per-dispatch overhead was paid only 6 times instead of 20.

Exact-match Redis caching (SHA-256 hash of image bytes, 60s TTL) was also implemented, but its benefit is conditional on repeated identical inputs, which are rare for this application.

3. Architecture Diagram

The diagram below maps the complete request flow of the optimized (Phase 3) system.

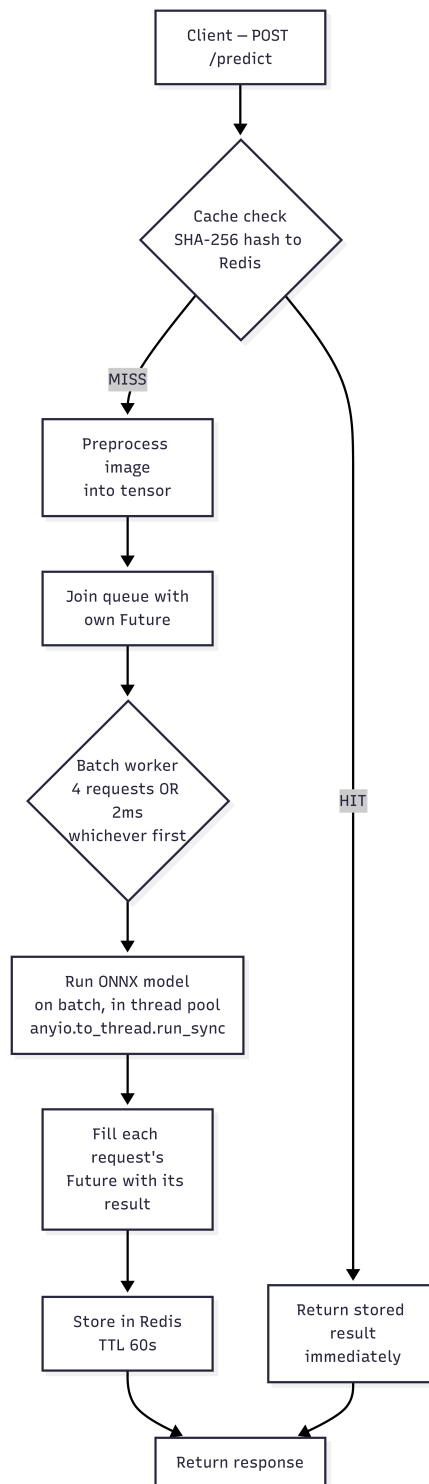


Figure 5. Optimized system request flow: cache check, batching queue, and thread-pool inference.